

Deep Residual Learning for Image Recognition

Generated by the MinionsOS Book $\rightarrow$ Paper compiler
(capability-validation artifact; reconstructed from an Agent-Native Research Artifact Book)

Abstract

Deeper neural networks are more difficult to train. As feed-forward convolutional networks for image recognition grow deeper, a counter-intuitive *degradation problem* emerges: adding layers to a suitably deep plain network increases not only test error but also *training* error, even with batch normalization and competent initialization, so the deeper model is no easier to optimize than its shallower counterpart. Here, we present a residual learning framework that eases the training of substantially deeper networks by reformulating each block to fit a residual mapping $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ and recovering the desired mapping through a parameter-free identity shortcut $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. We provide comprehensive empirical evidence that these residual networks are easier to optimize and gain accuracy from considerably increased depth: on ImageNet, residual nets reach a depth of up to 152 layers—eight times deeper than VGG nets yet of lower complexity—and a six-model ensemble attains 3.57% top-5 error on the test set, winning the first place in the ILSVRC 2015 classification task. We further analyze residual nets on CIFAR-10 with depths up to 110 and a 1202-layer stress test, and show that the learned representations transfer to object detection, where replacing a VGG-16 backbone with a 101-layer residual net yields a 28% relative improvement on COCO. The depth of representations is of central importance for visual recognition.

1 Introduction

Deep convolutional networks have driven a sequence of breakthroughs in image classification, where evidence suggests that network depth is of crucial importance and that features can be enriched by the number of stacked layers. A natural question follows: is learning better networks as easy as stacking more layers? An obstacle to answering it is that simply increasing depth does not monotonically help. When deeper networks are able to start converging, a *degradation problem* is exposed: as the depth increases, accuracy gets saturated and then degrades rapidly. Crucially, this degradation is not caused by overfitting—adding more layers to a suitably deep model leads to higher *training* error, as observed both on CIFAR-10 with plain networks of depth 20 through 110 and on ImageNet, where a 34-layer plain network obtains higher top-1 validation error than an 18-layer one despite having strictly greater capacity.

The degradation is counter-intuitive because a deeper model should be constructible from a shallower one with no loss: copy the shallower network and append identity layers, and the constructed deeper network has, by definition, training error no worse than its shallower source. That such a solution exists but is not found by stochastic gradient descent indicates that the difficulty lies in *optimization* rather than in representational capacity. The usual suspect—vanishing or exploding gradients—can be ruled out here: batch normalization [9] keeps forward-signal variances non-zero and backward gradient norms healthy, and the deep plain network still attains competitive accuracy, so the solver is making progress, only slowly. Existing remedies, including improved initialization [6, 8], batch normalization, intermediate auxiliary classifiers [13], and gated highway shortcuts [23, 24], allow networks of a few tens of layers to converge but have not demonstrated monotonic accuracy

gains at extreme depth: highway gates are data-dependent and carry parameters, and when a gate closes the layer behaves non-residually.

We address the *degradation problem* by introducing a deep residual learning framework. Instead of hoping a few stacked layers directly fit a desired underlying mapping $\mathcal{H}(\mathbf{x})$, we explicitly let these layers fit a residual mapping $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ and recover the original mapping as $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. The reformulation is motivated by the construction argument above and by the empirical observation that layer responses in residual nets are smaller in magnitude than in plain nets, consistent with the prior that optimal mappings in recognition tasks sit closer to identity than to zero. The addition is realized by a parameter-free *identity shortcut* that adds the block input to its output, so the residual variant has exactly the same number of parameters, depth, and width as the plain counterpart—isolating the effect of residual learning from any change in capacity. Identity shortcuts add neither parameters nor computational cost beyond an element-wise addition, and the entire network is trained end-to-end by ordinary stochastic gradient descent with backpropagation without modifying the optimizer.

Our experiments on the ImageNet 2012 classification dataset and on CIFAR-10, together with object-detection transfer experiments on PASCAL VOC and COCO, support the following contributions:

- **Diagnosis of the degradation problem.** We give controlled depth scans on both CIFAR-10 and ImageNet showing that deeper plain networks reach higher training and validation error than shallower ones under identical training, and argue—using batch normalization and the identity-construction argument—that this reflects an optimization difficulty rather than overfitting or vanishing gradients.
- **A parameter-free residual learning framework.** We reformulate stacked layers to fit a residual mapping with identity shortcuts, eliminating the degradation: at matched depth on ImageNet, the residual 34-layer net improves over both its plain counterpart and the shallower residual 18-layer net, and accuracy grows with depth up to 152 layers while the deepest model still has lower complexity than VGG nets.
- **Bottleneck designs and shortcut analysis at scale.** We show that three-layer bottleneck blocks make 50/101/152-layer networks practical at complexity comparable to a 34-layer net, that identity shortcuts suffice to fix degradation while projection shortcuts give only marginal gains, and that a six-model ensemble wins ILSVRC 2015 classification.
- **Generality of deep residual representations.** We demonstrate successful training on CIFAR-10 up to 110 layers (and a 1202-layer stress test that trains without optimization difficulty), and show that swapping a VGG-16 detection backbone for a 101-layer residual net yields large COCO and PASCAL VOC gains attributable solely to the learned representations.

2 Related Work

Residual representations. The idea of representing a signal by its residual with respect to a reference has a long history in vision and numerical computation. In image retrieval and classification, encoding residual vectors—for example, by quantizing residuals relative to a dictionary—has proven more effective than encoding original vectors, as in product quantization and related Fisher/VLAD encodings [10]. In low-level vision and graphics, multigrid and hierarchical-basis preconditioning solve partial differential equations far faster by reformulating the system in terms of residual variables at multiple scales [2, 26]. These methods establish a recurring principle—that a good reformulation or preconditioning can simplify optimization—but operate on hand-designed representations. We

carry the principle into deep learning by making every few stacked layers fit a residual function, so that the reformulation is learned rather than engineered.

Shortcut connections and gating. Skip and shortcut connections predate deep residual learning, appearing in early multilayer-perceptron design and in the auxiliary-classifier branches used to combat vanishing gradients [13]. The closest prior method is the highway network [23, 24], whose shortcuts are governed by data-dependent gates with their own parameters. Gating couples the shortcut to extra parameters and, more importantly, can “close”: when a gate approaches zero the layer reverts to a non-residual transformation, and highway networks have not shown accuracy gains beyond roughly one hundred layers. We adopt the general notion of a skip connection but reject the gating mechanism, using instead a parameter-free identity shortcut that is never closed and never adds parameters, which is what allows depth to be increased without bound on the optimization side.

Deep architectures and training of plain networks. A second line of work scales plain feed-forward depth directly. VGG nets establish the design philosophy of stacking small 3×3 convolutions under simple width rules [22], and we adopt these rules but reach far greater depth at much lower complexity. Inception/GoogLeNet networks add depth through a branching topology with internal shortcut-and-merge structure [25], which we use only as a comparison point rather than as an architectural ingredient. The ingredients that make very deep plain training feasible at all—batch normalization [9], variance-preserving initialization for rectifiers [6, 8], and rectified linear units [17]—are imported wholesale and applied identically to plain and residual variants, so that any difference between them is attributable to the residual reformulation and not to the training recipe. These techniques let tens-of-layer plain nets converge, but, as the long-recognized difficulty of training deep networks [1] would predict, they do not by themselves remove the *degradation problem*, which is precisely the gap residual learning closes.

Object detection frameworks. For the transfer experiments we build on the Faster R-CNN detection framework [18] together with the Fast R-CNN region classifier [4] and its R-CNN predecessor [5], keeping the detection pipeline—region-proposal network, RoI pooling, and four-step alternating training—fixed. To insert a fully-convolutional residual backbone that lacks hidden fully-connected layers, we follow the strategy of sharing full-image convolutional features and treating a late convolutional stage as the per-region classifier [19]. Holding the framework constant and changing only the backbone isolates the contribution of the learned representation, and we evaluate it on the standard PASCAL VOC [3], ImageNet [21], and COCO [15] benchmarks, with the CIFAR-10 analysis using the standard tiny-image dataset [11] and augmentation conventions established in prior recognition work [12]. Baselines from this literature, including network-in-network [14], maxout [7], and thin deep nets [20], anchor our CIFAR-10 comparison, and fully convolutional inference [16] underlies the multi-scale testing protocol.

3 Deep Residual Learning

3.1 Residual learning

Let $\mathcal{H}(\mathbf{x})$ denote an underlying mapping to be fit by a few stacked layers, with $\mathbf{x}$ the input to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate a complicated function, then it is equivalent to hypothesize that they can approximate the residual function $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ (assuming input and output have the same dimension). Rather than expecting the stacked layers to approximate $\mathcal{H}(\mathbf{x})$, we explicitly let them approximate $\mathcal{F}(\mathbf{x})$, and recover the original mapping as $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. Although both forms should be able to approximate the desired function asymptotically, the ease of learning differs. This reformulation is motivated by the

counter-intuitive degradation phenomenon discussed in section 1: if added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart, yet solvers struggle to find such a solution. With residual learning, if an identity mapping is optimal, the solver may simply drive the weights of the nonlinear layers toward zero to approach identity, rather than fit an identity by a stack of nonlinear layers. In practice the optimal functions are unlikely to be exactly identity, but the reformulation provides a better-conditioned starting point: if optimal functions are closer to identity than to zero, it is easier for the solver to find perturbations with reference to an identity mapping than to learn the function afresh.

3.2 Identity mapping by shortcuts

We adopt residual learning to every few stacked layers. A building block is defined as

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}, \quad (1)$$

where $\mathbf{x}$ and $\mathbf{y}$ are the input and output vectors of the layers considered and $\mathcal{F}(\mathbf{x}, \{W_i\})$ is the residual mapping to be learned. The operation $\mathcal{F} + \mathbf{x}$ is performed by a shortcut connection and an element-wise addition, followed by a second nonlinearity $\sigma(\mathbf{y})$. For a two-layer block, $\mathcal{F} = W_2 \sigma(W_1 \mathbf{x})$ with σ the rectified linear unit and biases omitted; a three-layer block applies $\mathcal{F} = W_3 \sigma(W_2 \sigma(W_1 \mathbf{x}))$. The shortcut connection in eq. (1) introduces neither extra parameters nor computational complexity beyond a negligible element-wise addition. This is not only attractive in practice but also central to the comparisons: the plain and residual networks then have the same number of parameters, depth, width, and computational cost, so a comparison between them is controlled.

Equation (1) requires $\mathcal{F}(\mathbf{x})$ and $\mathbf{x}$ to share dimensions. When they differ, for instance where the number of channels changes, a linear projection W_s is applied on the shortcut to match dimensions,

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s \mathbf{x}. \quad (2)$$

A square W_s may also be used in eq. (1), but experiments (section 4.2) show that identity shortcuts are sufficient for addressing the *degradation problem* and are economical, so the projection in eq. (2) is reserved for matching dimensions. The residual function $\mathcal{F}$ is flexible, but a single-layer $\mathcal{F}$ is similar to a linear layer and offers no observed advantage, so we use $\mathcal{F}$ with two or more layers.

3.3 Network architectures

We instantiate residual learning in a stage-structured convolutional network whose plain baseline follows VGG-style design rules: layers that produce feature maps of the same size keep the same number of filters, and when the spatial size is halved the number of filters is doubled so that the time complexity per layer is preserved. A single stem applies a 7×7 convolution with 64 filters at stride 2, followed by batch normalization, a rectified linear unit, and a 3×3 max-pool at stride 2, producing a $56 \times 56 \times 64$ feature map shared by every depth. Four residual stages (`conv2_x` through `conv5_x`) then stack building blocks; the first block of `conv3_x`, `conv4_x`, and `conv5_x` performs down-sampling with a stride-2 convolution rather than pooling, and the shortcut at those stage boundaries down-samples to match. A global average pool, a 1000-way fully-connected layer, and a softmax form the head. The residual network is obtained from the plain network by inserting shortcuts (eq. (1)) between every pair (basic block) or triple (bottleneck block) of layers, leaving the plain skeleton otherwise unchanged.

For the basic block used at 18 and 34 layers, $\mathcal{F}$ is two 3×3 convolutions. To reach greater depth economically we use a *bottleneck* block: a 1×1 convolution reduces the channel dimension,

Table 1: Architectures for ImageNet. Building blocks are shown in brackets with the number of stacked blocks. Down-sampling is performed by `conv3_1`, `conv4_1`, and `conv5_1` with a stride of 2. Source: Table 1 of the underlying artifact; supports the depth-scaling and bottleneck claims.

stage	output	18 / 34-layer (basic)	50 / 101 / 152-layer (bottleneck)	blocks (18/34/50/101/152)
<code>conv1</code>	112×112	7×7 , 64, stride 2		—
<code>conv2_x</code>	56×56	$[3 \times 3, 64] \times 2$	$[1 \times 1, 64; 3 \times 3, 64; 1 \times 1, 256]$	2 / 3 / 3 / 3 / 3
<code>conv3_x</code>	28×28	$[3 \times 3, 128] \times 2$	$[1 \times 1, 128; 3 \times 3, 128; 1 \times 1, 512]$	2 / 4 / 4 / 4 / 8
<code>conv4_x</code>	14×14	$[3 \times 3, 256] \times 2$	$[1 \times 1, 256; 3 \times 3, 256; 1 \times 1, 1024]$	2 / 6 / 6 / 23 / 36
<code>conv5_x</code>	7×7	$[3 \times 3, 512] \times 2$	$[1 \times 1, 512; 3 \times 3, 512; 1 \times 1, 2048]$	2 / 3 / 3 / 3 / 3
	1×1	average pool, 1000-d fc, softmax		—
FLOPs ($\times 10^9$)		1.8 (18), 3.6 (34)	3.8 (50), 7.6 (101), 11.3 (152)	VGG-19: 19.6

a 3×3 convolution operates on the reduced bottleneck, and a second 1×1 convolution restores the high-dimensional output, with the output width four times the bottleneck width. A bottleneck block carries the same per-block time complexity as the basic two-layer block while spanning three weighted layers, which is what makes 50/101/152-layer networks tractable. Identity shortcuts are especially important in bottleneck designs, because replacing the identity on the high-dimensional ends with a projection would roughly double both the time complexity and the model size of the block. Table 1 summarizes the per-depth stage layouts and floating-point complexity; the 34-layer model has the same complexity as its plain counterpart, and the 152-layer model, while eight times deeper than VGG-19, still has lower complexity.

For dimension-changing shortcuts we consider three options: Option A uses identity mapping with zero-padding for the increased channels and is entirely parameter-free; Option B uses projection shortcuts (eq. (2)) only where dimensions change and identity elsewhere; and Option C uses projections on all shortcuts. Section 4.2 quantifies their small differences. Within a stage, where dimensions do not change, identity shortcuts are always used. On CIFAR-10 we use a separate, deliberately minimal family to study depth in isolation: a first 3×3 convolution with 16 filters, followed by $2n$ layers at each of three feature-map sizes $\{32, 16, 8\}$ with widths $\{16, 32, 64\}$, giving $6n+2$ weighted layers, identity (Option A) shortcuts everywhere, and a global-average-pool/softmax head. Because the CIFAR shortcuts are all identity, the residual nets have *exactly* the same depth, width, and parameter count as their plain counterparts.

3.4 Implementation and training

We follow a single recipe across plain and residual variants so that comparisons are controlled. Batch normalization is applied right after each convolution and before activation; weights are initialized with the variance-preserving scheme for rectifiers; and no dropout is used. Networks are trained from scratch with stochastic gradient descent, momentum 0.9, and weight decay 10^{-4} . The complete forward pass of a single residual block is given in algorithm 1; the full network applies it stage by stage between the stem and the head. Design choices in this section are tied to the experimental verifications in section 4: the controlled plain-versus-residual comparison (section 4.1), the shortcut-option ablation (section 4.2), the bottleneck depth scan (section 4.3), and the CIFAR-10 and detection-transfer studies (sections 4.4 and 4.5).

Algorithm 1 Forward pass of one residual building block

```
1: function RESIDUALBLOCK( $\mathbf{x}$ ,  $\mathcal{F}$ , option, downsample)
2:   out  $\leftarrow \mathcal{F}(\mathbf{x})$   $\triangleright$  conv  $\rightarrow$  BN  $\rightarrow$  ReLU  $\rightarrow$  conv  $\rightarrow$  BN (or  $1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$ )
3:   if dim(out) = dim( $\mathbf{x}$ ) and not downsample then
4:     identity  $\leftarrow \mathbf{x}$   $\triangleright$  parameter-free identity shortcut
5:   else if option = A then
6:     identity  $\leftarrow$  ZEROPADCHANNELS(MAYBESTRIDE2( $\mathbf{x}$ ))
7:   else
8:     identity  $\leftarrow$  PROJECTION( $\mathbf{x}$ )  $\triangleright 1\times 1$  conv, optional stride 2
9:   end if
10:  out  $\leftarrow$  out + identity  $\triangleright$  element-wise addition
11:  return RELU(out)
12: end function
```

Table 2: Top-1 error (% , 10-crop testing) on ImageNet validation. The residual nets here have no extra parameter compared to their plain counterparts (Option A identity shortcuts). The deeper plain network degrades (34 worse than 18); the deeper residual net improves (34 better than 18). Source: Table 2.

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

4 Experiments

We evaluate residual learning on ImageNet 2012 classification (1.28M training images, 50k validation, 100k test, 1000 classes), on CIFAR-10 (50k training / 10k test 32×32 images, 10 classes), and on PASCAL VOC and COCO object detection. Unless stated otherwise, ImageNet results use 10-crop testing and all plain and residual variants share the single training recipe of section 3.4 so that every comparison is controlled.

4.1 Plain versus residual networks on ImageNet

We first compare 18- and 34-layer plain networks against their residual counterparts under identical training, where the residual nets add only parameter-free Option A identity shortcuts and therefore have no extra parameters (verifying the diagnosis and the elimination of the *degradation problem*). Table 2 reports the controlled comparison. The deeper plain network is *worse*: plain-34 reaches 28.54% top-1 error against 27.94% for plain-18. The training curves in the underlying analysis show that plain-34 sits above plain-18 throughout optimization, not merely at convergence, so the gap is a degradation of the optimization rather than overfitting; we further note that the deep plain network still reaches competitive accuracy, indicating the solver makes progress only slowly. Adding identity shortcuts reverses the ordering: ResNet-34 attains 25.03% top-1, which is 3.51 points better than plain-34 and 2.85 points better than ResNet-18 (27.88%). At 18 layers the plain and residual nets reach comparable final accuracy (27.94 vs. 27.88), but the residual variant converges faster from the start, indicating that residual learning eases optimization even where the plain net can ultimately converge. Together these results support both the existence of the *degradation problem* and its removal by residual learning.

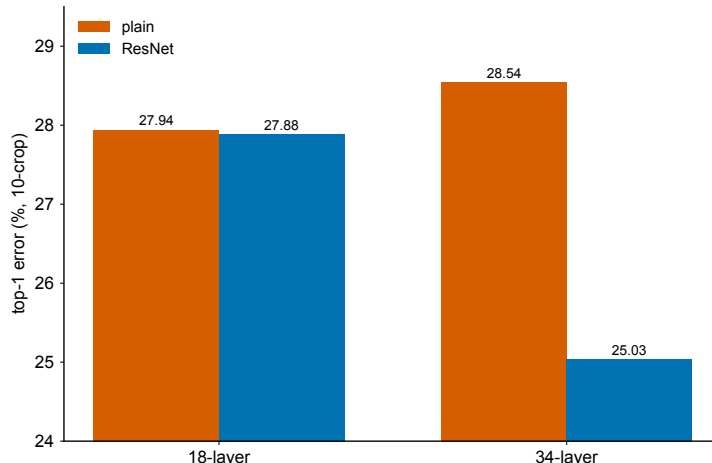


Figure 1: ImageNet top-1 validation error (% , 10-crop) for plain versus residual networks at 18 and 34 layers (Option A identity shortcuts, no extra parameters). The deeper plain network degrades (27.94 $\rightarrow$ 28.54) while the deeper residual net improves (27.88 $\rightarrow$ 25.03), reversing the ordering. Bars are the exact Table 2 values. $N = 1$ trained model per bar.

4.2 Identity versus projection shortcuts

We next ask whether the parameter-bearing projection shortcuts of eq. (2) are necessary, ablating the three shortcut options on ResNet-34 under the same recipe. Table 8 isolates the relevant rows. All three options improve over plain-34 by a wide margin (3.51 to 4.35 top-1 points), confirming that the gain comes from residual learning rather than from a particular shortcut form. The differences among the options are small: Option A (identity, parameter-free) reaches 25.03%, Option B (projection only where dimensions change) reaches 24.52%, and Option C (projection on every shortcut) reaches 24.19%. The slight B-over-A margin is attributable to the zero-padded extra channels in A carrying no residual learning, and the slight C-over-B margin to the parameters added by the 13 extra projection shortcuts. Because Option C is only marginally better yet costs extra parameters and memory, it is not essential for addressing the *degradation problem*; we therefore default to identity shortcuts and reserve projections for dimension-matching, which is especially important for the bottleneck designs where a projection on the high-dimensional ends would roughly double a block’s complexity.

4.3 Going deeper with bottleneck networks

Using the bottleneck block of section 3.3 with Option B shortcuts, we scale depth to 50, 101, and 152 layers and evaluate whether accuracy continues to improve with depth (verifying the depth-scaling and bottleneck claims). Table 3 reports the full 10-crop validation table. Top-1 error decreases monotonically with depth—22.85% (ResNet-50), 21.75% (ResNet-101), 21.43% (ResNet-152)—and every residual variant improves substantially over the 34-layer plain network (28.54%) and over the cross-family baselines VGG-16 (28.07 / 9.33) and PReLU-net (24.27 / 7.38). The gains come from depth rather than added parameters relative to the plain design, and from table 1 the 152-layer model has 11.3×10^9 FLOPs, still below VGG-19’s 19.6×10^9 despite being eight times deeper. Figure 2 plots the monotone trend for both metrics. Under the stronger single-model multi-scale fully-convolutional protocol (table 4), ResNet-152 reaches 19.38 / 4.49 top-1/top-5, surpassing every

Table 3: Error rates (% , 10-crop testing) on ImageNet validation. ResNet-50/101/152 use Option B (projections only for increasing dimensions). Top-1 error decreases monotonically with depth from 50 to 152, and every residual variant improves over the 34-layer plain network. Source: Table 3.

model	top-1 err.	top-5 err.
VGG-16 [22]	28.07	9.33
GoogLeNet [25]	—	9.15
PReLU-net [8]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 4: Single-model error rates (%) on ImageNet validation (multi-scale fully-convolutional testing); † denotes a result on the test set. The single-model ResNet-152 surpasses every prior single model. Source: Table 4.

method	top-1 err.	top-5 err.
VGG [22] (ILSVRC'14)	—	8.43 [†]
GoogLeNet [25] (ILSVRC'14)	—	7.89
VGG [22] (v5)	24.4	7.1
PReLU-net [8]	21.59	5.71
BN-inception [9]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

prior single model including PReLU-net (21.59 / 5.71) and BN-inception (21.99 / 5.81). Finally, a six-model ensemble attains 3.57% top-5 error on the ImageNet *test* set (table 5), improving on the best prior ensemble (BN-inception, 4.82%) and winning the first place in ILSVRC 2015 classification.

4.4 Analysis on CIFAR-10

To test whether residual learning generalizes beyond ImageNet and to probe extreme depth, we train the minimal $6n+2$ -layer CIFAR family at depths 20, 32, 44, 56, 110, and 1202, with plain nets of matched depth as controls (verifying the extreme-depth generalization and warmup claims). Table 6 and fig. 3 report test error. Residual error falls steadily with depth, from 8.75% at 20 layers to 6.43% at 110 layers (best; mean 6.61 ± 0.16 over five runs), while the matched plain nets degrade—plain-110 exceeds 60% error and is omitted from the comparison. The 110-layer residual net required a learning-rate warmup to begin converging cleanly: an initial rate of 0.1 was slightly too large at this depth, so we warmed up at 0.01 for roughly 400 iterations until training error fell below about 80%, then restored 0.1; this is a stability aid for very deep CIFAR nets rather than a fundamental requirement of residual learning. The 1202-layer net trains with no optimization difficulty (final training error below 0.1%), but its test error rises to 7.93%, higher than the 110-layer

Table 5: Ensemble error rates (% , top-5) on the ImageNet *test* set, reported by the test server. The six-model ResNet ensemble attains 3.57%, winning first place in ILSVRC 2015 classification. Source: Table 5.

method	top-5 err. (test)
VGG [22] (ILSVRC'14)	7.32
GoogLeNet [25] (ILSVRC'14)	6.66
VGG [22] (v5)	6.8
PReLU-net [8]	4.94
BN-inception [9]	4.82
ResNet (ILSVRC'15)	3.57

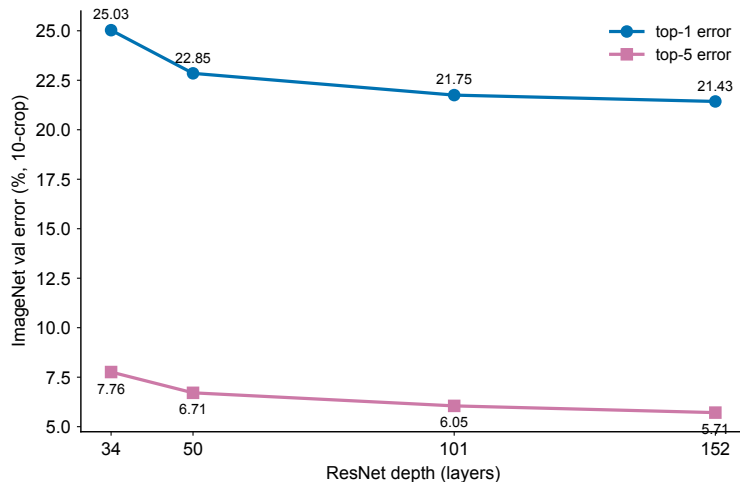


Figure 2: ImageNet validation error (% , 10-crop) versus residual depth; ResNet-34 uses Option A and the deeper nets use Option B. Both top-1 and top-5 error fall monotonically as depth grows from 34 to 152 (**top-1** 25.03 $\rightarrow$ 21.43). Markers are the exact Table 3 values; no curve is interpolated. $N = 1$ trained model per depth.

net. Because optimization is not the bottleneck at 1202 layers, we attribute this to overfitting: a 19.4M-parameter model on a 50k-image dataset, trained here without strong regularization such as maxout or dropout.

4.5 Object-detection transfer

Finally, we test whether deeper residual representations transfer to localization-sensitive tasks by replacing the VGG-16 backbone of a baseline Faster R-CNN detector with ResNet-101, holding the detection framework and hyperparameters fixed (verifying the transfer claim). Table 7 reports the swap. On PASCAL VOC the mAP@.5 rises from 73.2 to 76.4 on VOC07 and from 70.4 to 73.8 on VOC12. On COCO the mAP@.5 improves from 41.5 to 48.4, and the stricter mAP@[.5,.95]—which rewards tighter localization—improves from 21.2 to 27.2, a 6.0 point absolute and 28% relative gain. That the mAP@[.5,.95] gain is comparable to the mAP@.5 gain indicates the deeper features help both recognition and localization, and because only the backbone changed, the improvement is attributable to the learned representation.

Table 6: Classification error (%) on the CIFAR-10 test set; all methods use data augmentation. For ResNet-110 we report best (mean $\pm$ std) over five runs. Error falls as residual depth grows from 20 to 110; the 1202-layer net trains successfully but overfits. Source: Table 6.

method	# layers	# params	error (%)
Maxout [7]	—	—	9.38
NIN [14]	—	—	8.81
DSN [13]	—	—	8.22
FitNet [20]	19	2.5M	8.39
Highway [23, 24]	19	2.3M	7.54 (7.72 $\pm$ 0.16)
Highway [23, 24]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61 $\pm$ 0.16)
ResNet	1202	19.4M	7.93

Table 7: Object-detection transfer with baseline Faster R-CNN: swapping a VGG-16 backbone for ResNet-101 with otherwise identical hyperparameters. PASCAL VOC reports mAP@.5 (VOC07 test from 07+12 training; VOC12 test from 07++12 training); COCO reports mAP@.5 and mAP@[.5,.95] on the validation set. The COCO mAP@[.5,.95] gain of 6.0 points is a 28% relative improvement. Source: Tables 7 and 8.

backbone	PASCAL VOC (mAP@.5)		COCO val	
	VOC07 test	VOC12 test	mAP@.5	mAP@[.5,.95]
VGG-16	73.2	70.4	41.5	21.2
ResNet-101	76.4	73.8	48.4	27.2

5 Conclusion

We have shown that the difficulty of training very deep convolutional networks is largely a problem of optimization, not of representational capacity, and that reframing each block to learn a residual with respect to its input—recovered through a parameter-free identity shortcut—makes depth a usable resource rather than a liability. The reformulation removes the degradation that afflicts plain networks, lets accuracy grow with depth far beyond what was previously trainable, and does so without adding parameters, auxiliary losses, or a specialized optimizer. The same deep representations, learned for classification, transfer to object detection and improve localization-sensitive metrics, indicating that the depth of representations is of central importance across visual recognition tasks rather than a classification-specific artifact.

Limitations

Several limitations bound these conclusions. First, the claim that residual learning eases *optimization* is supported empirically—by the controlled depth comparisons and by the observation that residual layer responses are smaller in magnitude than plain ones—but is not formally proven; the argument rests on the hypothesis that stacked nonlinear layers can asymptotically approximate the desired residual functions, which remains an open question. Vanishing gradients and longer training were

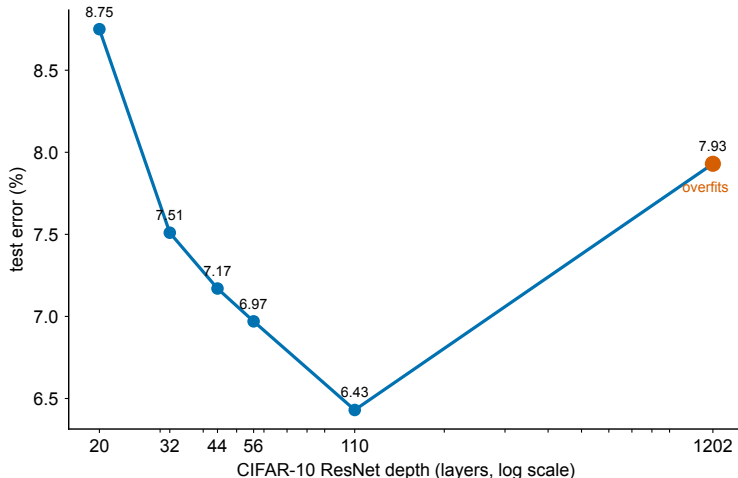


Figure 3: CIFAR-10 test error (%) versus residual depth (log scale; Option A identity shortcuts). Error decreases monotonically through 110 layers ($8.75 \rightarrow 6.43$) and then rises at 1202 layers (highlighted), where the network still trains cleanly but overfits the small dataset. Markers are the exact Table 6 values; the 110-layer point is the best of five runs.

Table 8: ResNet-34 shortcut-option ablation on ImageNet validation (% , 10-crop), isolated from Table 3 with the plain-34 baseline. All three options beat plain-34 by a wide margin; the differences among A, B, and C are small (≤ 0.84 top-1). Source: derived subset of Table 3.

model	shortcut option	top-1 err.	top-5 err.	Δ top-1 vs. plain-34
plain-34	none	28.54	10.02	—
ResNet-34 A	identity + zero-pad	25.03	7.76	−3.51
ResNet-34 B	projection on dim. change	24.52	7.46	−4.02
ResNet-34 C	projection on every shortcut	24.19	7.40	−4.35

examined and set aside as explanations: with batch normalization the forward signals do not vanish, and training plain networks for three times as many iterations did not close the degradation gap, so neither accounts for the phenomenon, but ruling out these specific causes is not the same as a positive proof of the optimization-landscape claim. Second, gains at extreme depth are not unconditional: on the small CIFAR-10 dataset the 1202-layer network optimizes cleanly yet overfits relative to the 110-layer one, and this work applies no strong regularization to counter it. Third, the architecture exploration is limited: a single-layer residual function offers no advantage and is excluded, only depth (not width) is scaled, and the solver studied is stochastic gradient descent with momentum, with no comparison to second-order methods. Finally, the headline results depend on testing protocols that should not be mixed—10-crop and multi-scale fully-convolutional testing report different absolute numbers, and the strongest aggregate result is a multi-model ensemble that a single model does not reproduce.

Future Work

Two concrete directions follow. The extreme-depth overfitting on small data invites combining residual networks with explicit regularizers such as maxout or dropout, to test whether the 1202-layer

model’s clean optimization can be converted into a test-error improvement rather than a regression. Separately, the residual-response analysis motivates studying stronger or explicit regularization on the residual functions themselves and characterizing how per-layer response magnitude scales with depth, toward a predictive rather than merely descriptive account of why residual reformulation eases optimization.

References

- [1] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 1994.
- [2] William L. Briggs, Van Emden Henson, and Steve F. McCormick. A multigrid tutorial. 2000.
- [3] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The PASCAL visual object classes (VOC) challenge. *International Journal of Computer Vision (IJCV)*, 2010.
- [4] Ross Girshick. Fast R-CNN. In *International Conference on Computer Vision (ICCV)*, 2015.
- [5] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.
- [6] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.
- [7] Ian J. Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In *International Conference on Machine Learning (ICML)*, 2013.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *International Conference on Computer Vision (ICCV)*, 2015.
- [9] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.
- [10] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. 2011.
- [11] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2012.
- [13] Chen-Yu Lee, Saining Xie, Patrick Gallagher, Zhengyou Zhang, and Zhuowen Tu. Deeply-supervised nets. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2015.

- [14] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.
- [15] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, 2014.
- [16] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [17] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *International Conference on Machine Learning (ICML)*, 2010.
- [18] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2015.
- [19] Shaoqing Ren, Kaiming He, Ross Girshick, Xiangyu Zhang, and Jian Sun. Object detection networks on convolutional feature maps. *arXiv preprint arXiv:1504.06066*, 2015.
- [20] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. In *International Conference on Learning Representations (ICLR)*, 2015.
- [21] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV)*, 2015.
- [22] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2015.
- [23] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.
- [24] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. In *Advances in Neural Information Processing Systems (NIPS)*, 2015. arXiv:1507.06228.
- [25] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [26] Richard Szeliski. Locally adapted hierarchical basis preconditioning. *ACM Transactions on Graphics (SIGGRAPH)*, 2006.